Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №6

по дисциплине «Логика и основы алгоритмизации в инженерных задачах»

на тему: «Унарные и бинарные операции над графами»

Выполнили ст.гр. 25ВВВ3:
Волков И.Р.
Итянина В.А.
Приняли:
к.т.н., доцент Юрова О.В.
к.т.н, ст. преподаватель, Деев М.В.

Пенза 2026

Цель работы: изучить унарные и бинарные операции над графами, научится реализовывать операции отождествления вершин, стягивания ребра, расщепления вершины, объединения, пересечения, кольцевой суммы и декартова произведения графов.

Ход работы

Унарные и бинарные операции над графами – это действия, с помощью которых можно изменять граф или получать новый граф из одного или нескольких исходных графов.

- Унарная операция – выполняется над одним графом

- Удаление вершины – из графа убирают одну вершину и все ребра, связанные с ней.
- Удаление ребра – из графа убирают одно или несколько рёбер.
- Добавление вершины или ребра – в граф вносят новые элементы.

- Бинарная операция – выполняется над двумя графами

- Объединение графов (объединяем вершины и рёбра обоих графов, если вершина или ребро встречается в обоих графах, оно учитывается один раз)
- Пересечение графов (оставляем только те вершины и рёбра, которые есть в обоих графах)
- Разность графов (оставляем элементы первого графа, которых нет во втором)
- Концевая сумма графов (оставляем рёбра, которые присутствуют только в одном из двух графов, но не в обоих одновременно)

Отличия:

Признак	**Унарная операция**	**Бинарная операция**
Сколько исходных графов?	Один	Два
Пример	Удаление вершины	Объединение графов
Что происходит?	Изменяем один граф	Сопоставляем или объединяем два графа

Задание 1: сгенерируйте (используя генератор случайных чисел) две матрицы M_1 и M_2 смежности неориентированных помеченных графов G_1, G_2. Выведите сгенерированные матрицы на экран. *Для указанных графов преобразуйте представление матриц смежности в списки смежности. Выведите полученные списки на экран.

Код:

```c
#define _CRT_SECURE_NO_WARNINGS
#define _CRT_NONSTDC_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <locale.h>

#define MAXV 100
#define MAXNAME 32

void print_matrix(int n, int mat[MAXV][MAXV], char
names[MAXV][MAXNAME]) {
    printf("Матрица смежности (%d вершин):\n", n);

    printf("%6s", "");
    for (int j = 0; j < n; j++) printf("%6s", names[j]);
    printf("\n");

    for (int i = 0; i < n; i++) {
        printf("%6s", names[i]);
        for (int j = 0; j < n; j++) printf("%6d", mat[i][j]);
        printf("\n");
    }
}

void print_adj_list(int n, int mat[MAXV][MAXV], char
names[MAXV][MAXNAME]) {
    printf("Списки смежности:\n");
    for (int i = 0; i < n; i++) {
        printf("%s: ", names[i]);
        int first = 1;
        for (int j = 0; j < n; j++) {
            if (mat[i][j]) {
                if (!first) printf(", ");
                printf("%s", names[j]);
                first = 0;
            }
        }
        printf("\n");
    }
}

void generate_graph(int n, int mat[MAXV][MAXV], double p) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) mat[i][j] = 0;

    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if ((double)rand() / RAND_MAX < p) {
                mat[i][j] = 1;
```

```
                mat[j][i] = 1;
            }
}

int main() {
    setlocale(LC_ALL, "Russian");
    srand((unsigned)time(NULL));

    int n;
    printf("Введите число вершин для G1 и G2: ");
    scanf_s("%d", &n);
    if (n <= 0 || n > MAXV) {
        printf("Недопустимое число вершин. Допустимо от 1 до
%d.\n", MAXV);
        return 1;
    }

    int M1[MAXV][MAXV], M2[MAXV][MAXV];
    char names[MAXV][MAXNAME];

    for (int i = 0; i < n; i++)
        snprintf(names[i], MAXNAME, "v%d", i + 1);

    generate_graph(n, M1, 0.5);
    generate_graph(n, M2, 0.5);

    printf("\n Граф G1 \n");
    print_matrix(n, M1, names);
    printf("\n");
    print_adj_list(n, M1, names);

    printf("\n Граф G2 \n");
    print_matrix(n, M2, names);
    printf("\n");
    print_adj_list(n, M2, names);

    return 0;
}
```

Результат работы программы представлен на рисунке 1.

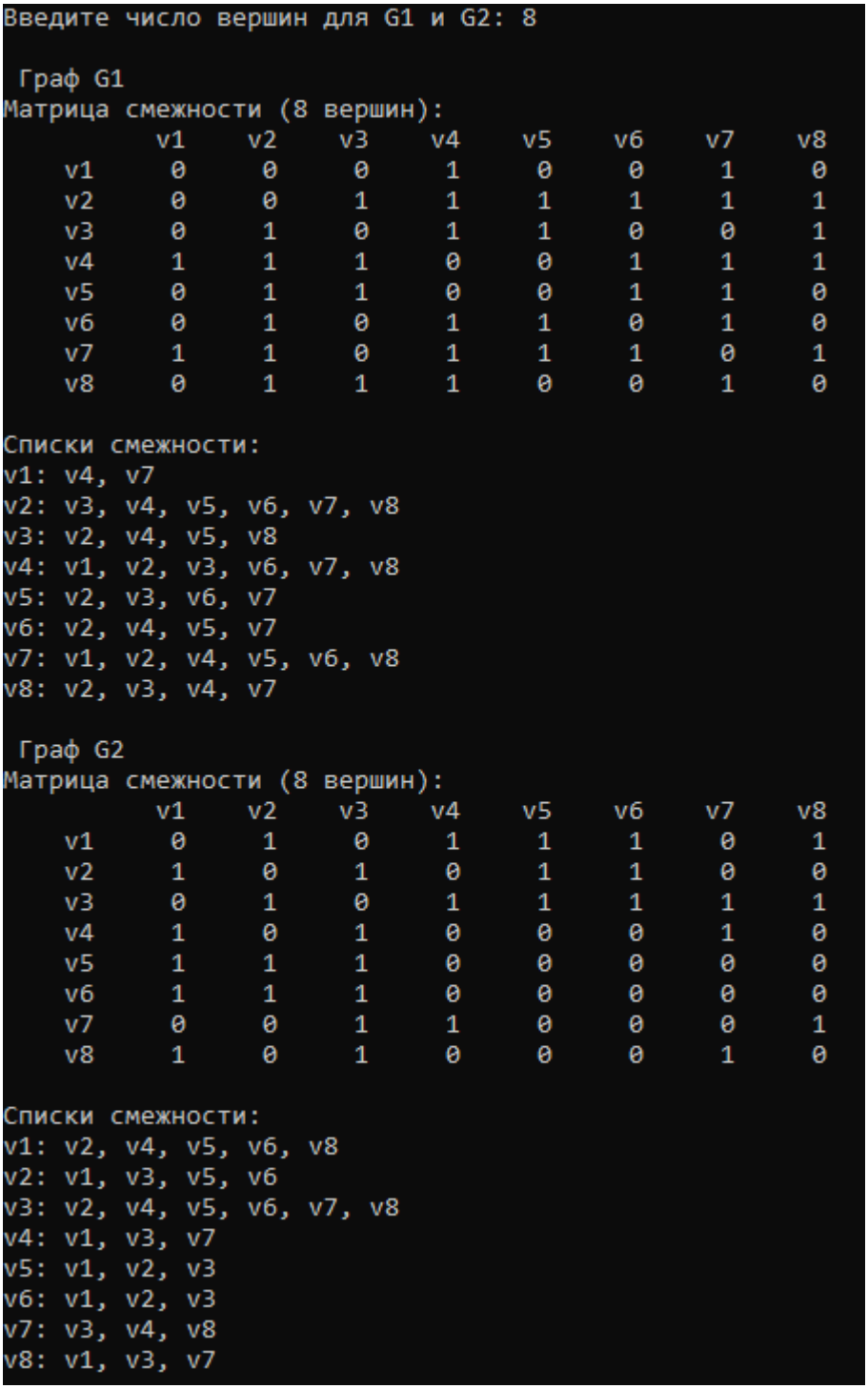


Рис.1 – результат работы программы к заданию 1.

Задание 2: для матричной формы представления графов выполните операцию: 1. отождествление вершин; 2. стягивание ребра; 3. расщепления вершины. Номера выбираемых для выполнения операции вершин ввести с клавиатуры. Результат выполнение операции выведите на экран. *Для преставления графов в виде списков смежности выполните операцию: 1. отождествления вершин; 2. стягивания ребра; 3. расщепления вершины. Номера выбираемых для выполнения операции вершин ввести с клавиатуры. Результат выполнения выведите на экран.

Код:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <locale.h>

#define MAXV 100
#define MAXNAME 32

void print_matrix(int n, int mat[MAXV][MAXV], char
names[MAXV][MAXNAME]) {
    printf("Матрица смежности (%d вершин):\n", n);
    printf("%6s", "");
    for (int j = 0; j < n; j++) printf("%8s", names[j]);
    printf("\n");
    for (int i = 0; i < n; i++) {
        printf("%6s", names[i]);
        for (int j = 0; j < n; j++) printf("%8d", mat[i][j]);
        printf("\n");
    }
}

void print_adj_list(int n, int mat[MAXV][MAXV], char
names[MAXV][MAXNAME]) {
    printf("Списки смежности:\n");
    for (int i = 0; i < n; i++) {
        printf("%s: ", names[i]);
        int first = 1;
        for (int j = 0; j < n; j++) {
            if (mat[i][j]) {
                if (!first) printf(", ");
                printf("%s", names[j]);
                first = 0;
            }
        }
        printf("\n");
    }
}

void generate_graph(int n, int mat[MAXV][MAXV], double p) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) mat[i][j] = 0;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if ((double)rand() / RAND_MAX < p)
                mat[i][j] = mat[j][i] = 1;
}

void identify_vertices(int n, int mat[MAXV][MAXV], char
names[MAXV][MAXNAME],
```

```
    int u, int v, int* nn, int res[MAXV][MAXV], char
res_names[MAXV][MAXNAME]) {
    if (u == v) { printf("Вершины должны быть различны!\n"); *nn
= -1; return; }

    int map[MAXV];
    int idx = 0;
    for (int i = 0; i < n; i++) {
        if (i == u || i == v) map[i] = -1;
        else map[i] = idx++;
    }
    int z = idx;
    int total = idx + 1;

    for (int i = 0; i < total; i++)
        for (int j = 0; j < total; j++) res[i][j] = 0;

    for (int i = 0; i < n; i++) {
        if (map[i] != -1) strcpy(res_names[map[i]], names[i]);
    }
    snprintf(res_names[z], MAXNAME, "%s%s", names[u], names[v]);

    for (int i = 0; i < n; i++) {
        if (map[i] == -1) continue;
        for (int j = 0; j < n; j++) {
            if (map[j] == -1) continue;
            if (mat[i][j]) res[map[i]][map[j]] = 1;
        }
    }

    for (int w = 0; w < n; w++) {
        if (map[w] == -1) continue;
        if (mat[w][u] || mat[w][v]) {
            res[map[w]][z] = 1;
            res[z][map[w]] = 1;
        }
    }

    if (mat[u][v] || mat[u][u] || mat[v][v])
        res[z][z] = 1;

    *nn = total;
}

void contract_edge(int n, int mat[MAXV][MAXV], char
names[MAXV][MAXNAME],
    int u, int v, int* nn, int res[MAXV][MAXV], char
res_names[MAXV][MAXNAME]) {
    if (u == v) { printf("Вершины должны быть различны!\n"); *nn
= -1; return; }
    if (mat[u][v] == 0) { printf("Ребра между вершинами
нет!\n"); *nn = -1; return; }
```

```
    int tmp[MAXV][MAXV];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) tmp[i][j] = mat[i][j];
    tmp[u][v] = 0;
    tmp[v][u] = 0;

    identify_vertices(n, tmp, names, u, v, nn, res, res_names);

    if (*nn > 0) {
        int z = *nn - 1;
        snprintf(res_names[z], MAXNAME, "(%s %s)", names[u],
names[v]);
    }
}

void split_vertex(int n, int mat[MAXV][MAXV], char
names[MAXV][MAXNAME],
    int v, int* nn, int res[MAXV][MAXV], char
res_names[MAXV][MAXNAME]) {
    int total = n + 1, w = n;

    for (int i = 0; i < total; i++)
        for (int j = 0; j < total; j++) res[i][j] = 0;

    for (int i = 0; i < n; i++) {
        strcpy(res_names[i], names[i]);
        for (int j = 0; j < n; j++)
            res[i][j] = mat[i][j];
    }

    snprintf(res_names[w], MAXNAME, "%s*", names[v]);

    for (int i = 0; i < n; i++) {
        if (mat[v][i]) {
            res[w][i] = 1;
            res[i][w] = 1;
        }
    }

    res[v][w] = 1;
    res[w][v] = 1;

    *nn = total;
}

int main() {
    setlocale(LC_ALL, "Russian");
    srand((unsigned)time(NULL));

    int n;
    printf("Введите число вершин для G1 и G2: ");
    scanf_s("%d", &n);
```

```
    if (n <= 0 || n > MAXV) { printf("Неверное число!\n");
return 1; }

    int G1[MAXV][MAXV], G2[MAXV][MAXV];
    char names1[MAXV][MAXNAME], names2[MAXV][MAXNAME];

    for (int i = 0; i < n; i++) {
        snprintf(names1[i], MAXNAME, "v%d", i + 1);
        snprintf(names2[i], MAXNAME, "v%d", i + 1);
    }

    generate_graph(n, G1, 0.5);
    generate_graph(n, G2, 0.5);

    printf("\n Исходный граф G1 \n");
    print_matrix(n, G1, names1);
    print_adj_list(n, G1, names1);

    printf("\n Исходный граф G2 \n");
    print_matrix(n, G2, names2);
    print_adj_list(n, G2, names2);

    int op, g, u, v, nn;
    int res[MAXV][MAXV];
    char res_names[MAXV][MAXNAME];

    while (1) {
        printf("\n МЕНЮ \n");
        printf(" 1 - отождествление вершин\n");
        printf(" 2 - стягивание ребра\n");
        printf(" 3 - расщепление вершины\n");
        printf(" 4 - показать исходные G1 и G2\n");
        printf(" 0 - выход\n");
        printf("Выбор операции: ");
        scanf_s("%d", &op);
        if (op == 0) break;

        if (op == 4) {
            printf("\n G1 \n");
            print_matrix(n, G1, names1);
            print_adj_list(n, G1, names1);
            printf("\n G2 \n");
            print_matrix(n, G2, names2);
            print_adj_list(n, G2, names2);
            continue;
        }

        printf("К какому графу применить? (1-G1, 2-G2): ");
        scanf_s("%d", &g);
        if (g != 1 && g != 2) { printf("Неверно!\n"); continue;
}

        int (*G)[MAXV] = (g == 1) ? G1 : G2;
```

```
        char (*nm)[MAXNAME] = (g == 1) ? names1 : names2;

        if (op == 1) {
            printf("Введите номера вершин u v для отождествления
(1..%d): ", n);
            scanf_s("%d %d", &u, &v);
            if (u < 1 || u > n || v < 1 || v > n) {
printf("Ошибка!\n"); continue; }
            identify_vertices(n, G, nm, u - 1, v - 1, &nn, res,
res_names);
        }
        else if (op == 2) {
            printf("Введите вершины u v для стягивания ребра
(1..%d): ", n);
            scanf_s("%d %d", &u, &v);
            if (u < 1 || u > n || v < 1 || v > n) {
printf("Ошибка!\n"); continue; }
            contract_edge(n, G, nm, u - 1, v - 1, &nn, res,
res_names);
        }
        else if (op == 3) {
            printf("Введите номер вершины для расщепления
(1..%d): ", n);
            scanf_s("%d", &v);
            if (v < 1 || v > n) { printf("Ошибка!\n"); continue;
}
            split_vertex(n, G, nm, v - 1, &nn, res, res_names);
        }
        else {
            printf("Неверная операция!\n");
            continue;
        }

        if (nn > 0) {
            printf("\nРезультат:\n");
            print_matrix(nn, res, res_names);
            print_adj_list(nn, res, res_names);
        }
    }
    return 0;
}
```

Результат работы программы представлен на рисунке 2.

```
Введите число вершин для G1 и G2: 7

 Исходный граф G1
Матрица смежности (7 вершин):
          v1      v2      v3      v4      v5      v6      v7
   v1      0       0       1       0       0       0       1
   v2      0       0       1       0       1       1       0
   v3      1       1       0       0       0       1       1
   v4      0       0       0       0       0       0       0
   v5      0       1       0       0       0       1       0
   v6      0       1       1       0       1       0       1
   v7      1       0       1       0       0       1       0
Списки смежности:
v1: v3, v7
v2: v3, v5, v6
v3: v1, v2, v6, v7
v4:
v5: v2, v6
v6: v2, v3, v5, v7
v7: v1, v3, v6

 Исходный граф G2
Матрица смежности (7 вершин):
          v1      v2      v3      v4      v5      v6      v7
   v1      0       0       0       0       1       1       1
   v2      0       0       0       1       1       0       0
   v3      0       0       0       1       0       1       0
   v4      0       1       1       0       0       0       1
   v5      1       1       0       0       0       1       0
   v6      1       0       1       0       1       0       0
   v7      1       0       0       1       0       0       0
Списки смежности:
v1: v5, v6, v7
v2: v4, v5
v3: v4, v6
v4: v2, v3, v7
v5: v1, v2, v6
v6: v1, v3, v5
v7: v1, v4

 МЕНЮ
 1 - отождествление вершин
 2 - стягивание ребра
 3 - расщепление вершины
 4 - показать исходные G1 и G2
 0 - выход
Выбор операции: 2
К какому графу применить? (1-G1, 2-G2): 2
Введите вершины u v для стягивания ребра (1..7): 1 5
```

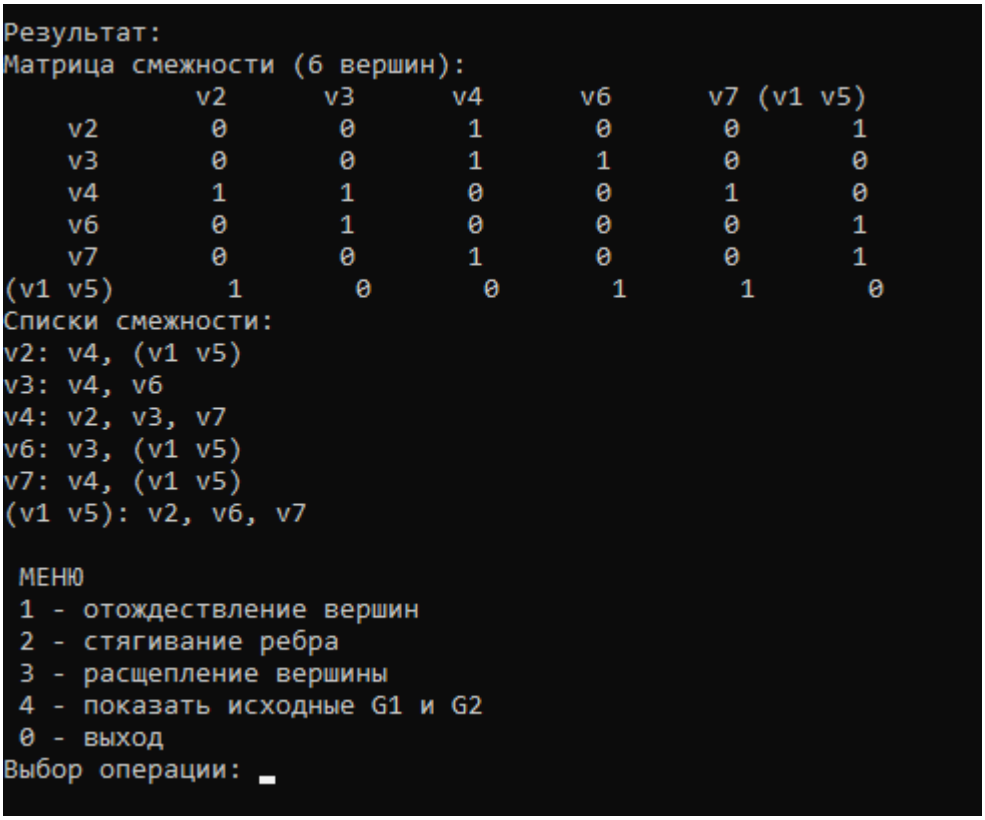


Рис.2 – результат работы программы к заданию 2.

Задание 3: для матричной формы представления графов выполните операцию: 1. объединение $G = G_1 G_2$; 2. пересечения $G = G_1 G_2$; 3. концевой суммы $G = G_1 + G_2$. Результат выполнения операции выведите на экран.

Код:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

int main() {
    setlocale(LC_ALL, "Russian");
    srand(time(NULL));
    int n, G1[50][50], G2[50][50], R[50][50];

    printf("Введите n: ");
    scanf_s("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            G1[i][j] = G2[i][j] = 0;

    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) {
            if (rand() % 2) G1[i][j] = G1[j][i] = 1;
            if (rand() % 2) G2[i][j] = G2[j][i] = 1;
        }
```

```
    printf("\nG1:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) printf("%d ", G1[i][j]);
        printf("\n");
    }
    printf("G2:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) printf("%d ", G2[i][j]);
        printf("\n");
    }

    printf("\nОбъединение:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            R[i][j] = G1[i][j] | G2[i][j];
            printf("%d ", R[i][j]);
        }
        printf("\n");
    }

    printf("\nПересечение:\n");
    int minN = n - 1;
    for (int i = 0; i < minN; i++) {
        for (int j = 0; j < minN; j++) {
            R[i][j] = G1[i][j] & G2[i][j];
            printf("%d ", R[i][j]);
        }
        printf("\n");
    }

    printf("\nКольцевая сумма:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            R[i][j] = G1[i][j] ^ G2[i][j];
            printf("%d ", R[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Результат работы программы представлен на рисунке 3.

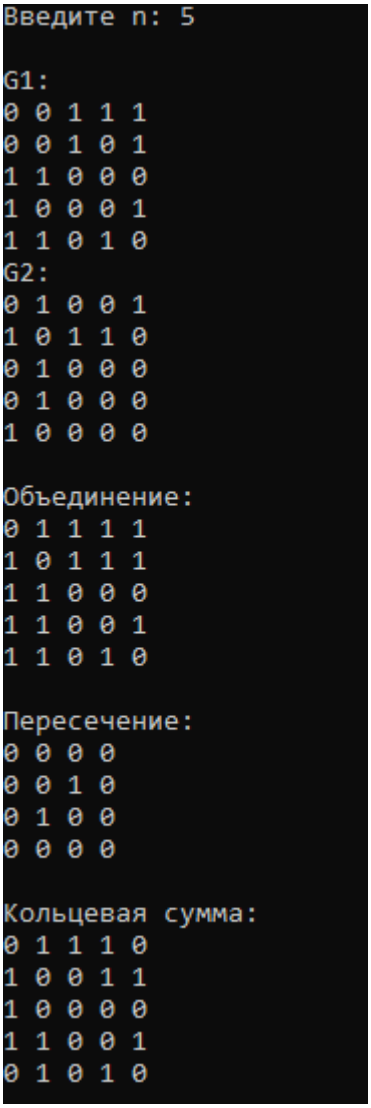


Рис.3– результат работы программы к заданию 3.

Вывод: в ходе лабораторной работы были сгенерированы два графа (G_1 и G_2), построены их матрицы смежности и списки смежности. Для графов выполнены унарные операции: отождествление вершин, стягивание ребра и расщепление вершины, а также бинарные операции: объединение, пересечение, кольцевая сумма и декартово произведение.